LLM LoRA / QLoRA Fine-Tuning

Technical Project Report

Parameter-efficient adaptation of Qwen2.5-7B with empirical rank comparison, controlled inference benchmarks, and production-oriented serving

Validated runtime	Configuration
Compute	NVIDIA A30 24 GB | SLURM GPU node `g-0`
Software	Python 3.11.11 | PyTorch 2.11.0+cu128 | Transformers 5.17.0
Base model	Qwen/Qwen2.5-7B (base, not instruction-tuned)
Dataset	databricks/databricks-dolly-15k
Adaptation	QLoRA, 4-bit NF4, BF16 compute, double quantization
Rank study	r8 vs r16; selected runtime adapter: QLoRA r8
Final validation	1,471 examples | r8 loss 1.592670 | perplexity 4.9169
Serving	FastAPI + LangGraph + browser frontend | CUDA inference
Observability	Application logging + Prometheus exposition endpoint
Final tests	6 passed, including real GPU end-to-end API inference

Scope note: this is a hands-on end-to-end portfolio project designed to demonstrate parameter-efficient LLM adaptation, evaluation, runtime trade-off analysis, and production-oriented ML/LLM engineering practices. It is not presented as long-term commercial production experience.

September 2026

1. Executive summary

This project adapts the 7.6B-parameter Qwen2.5 base model for instruction-following with QLoRA, compares two LoRA ranks under the same explicit hyperparameter configuration, evaluates memory and validation behavior, benchmarks multiple inference representations, and then wraps the selected adapter in a production-oriented application layer.

The central engineering result is not simply that a LoRA adapter can be trained. The project makes the trade-offs measurable:

- r8 and r16 reach almost identical validation loss, while r8 uses half the trainable LoRA parameters and a smaller adapter artifact;
- 4-bit model loading reduces GPU allocation substantially, but on the A30 used here it is not automatically faster than BF16 inference;
- keeping PEFT active at inference introduces measurable overhead, while a merged BF16 model recovers approximately native BF16-base throughput;
- the selected QLoRA r8 adapter is served through a FastAPI + LangGraph architecture with health/readiness separation, metrics, logging, a browser frontend, and real GPU end-to-end tests.

The final project therefore demonstrates the full path from data preparation and PEFT mechanics to deployment-oriented runtime design, without conflating a portfolio implementation with commercial production ownership.

2. Problem, objective, and scope

The objective was to study parameter-efficient fine-tuning as an engineering system rather than as a single training call. The project asks: how much adaptation is needed, what memory is saved, what changes when LoRA rank doubles, how does quantization affect deployment choices, what overhead does an active PEFT adapter introduce, and how should the result be exposed through a clean application interface?

The experimental scope is intentionally bounded: one base model, one dataset, two LoRA ranks, one primary GPU architecture, deterministic generation for comparisons, and a small qualitative sample. Conclusions are therefore project-specific rather than universal claims about LoRA or QLoRA.

3. Base model and dataset

3.1 Base model

The base checkpoint is `Qwen/Qwen2.5-7B`, specifically the base model rather than the instruction-tuned variant. This was deliberate: instruction adaptation is easier to observe when the starting checkpoint has not already been instruction-aligned.

Property	Verified value
Architecture	Qwen2ForCausalLM
Logical parameters	7,615,616,512
Transformer layers	28
Hidden size	3,584
Attention heads / KV heads	28 / 4
Intermediate size	18,944
Base dtype	bfloat16
Tokenizer EOS / PAD	`<|endoftext|>` / `<|endoftext|>`

Table 1: *Verified base-model characteristics used by the project.*

The LoRA target inspection confirmed attention projection shapes consistent with grouped-query attention: `q_proj` and `o_proj` project 3584 to 3584, while `k_proj` and `v_proj` project 3584 to 512.

3.2 Dataset

The project uses `databricks/databricks-dolly-15k`, an instruction dataset with the fields `instruction`, `context`, `response`, and `category`. The raw dataset contains 15,011 examples across brainstorming, classification, closed QA, creative writing, general QA, information extraction, open QA, and summarization.

A stratified split with seed 42 produced 13,509 training examples and 1,502 validation examples before length filtering.

4. Data preparation and supervision

The training prompt preserves the dataset distinction between instructions with and without context.

```
### Instruction:
{instruction}

### Context:
{context}

### Response:
{response}
```

Without context, the context block is omitted. Tokenization constructs `input_ids = prompt + response + EOS`; prompt labels are set to -100, so only the response and EOS contribute to the supervised loss.

Token-length analysis showed a long tail (p99 = 1,163 tokens; maximum = 8,601). A 1,024-token maximum was selected. Instead of truncating, overlength examples were filtered so that context-heavy examples would not silently lose evidence.

Split	Before	After	Removed	Removed %
Train	13,509	13,334	175	1.295%
Validation	1,502	1,471	31	2.064%

Table 2: *Length filtering at 1,024 tokens.*

Dynamic batch collation pads only to the longest sequence in each batch; token padding uses the tokenizer pad ID, attention padding uses 0, and label padding uses -100.

5. LoRA and QLoRA design

For a frozen linear projection $W \in \mathbb{R}^{out \times in}$, LoRA adds low-rank matrices $A \in \mathbb{R}^{r \times in}$ and $B \in \mathbb{R}^{out \times r}$. The trainable count contributed by that adapted layer is $r(in + out)$.

The project adapts `q_proj`, `k_proj`, `v_proj`, and `o_proj`. Shared settings are LoRA dropout 0.05, no bias, and $\alpha/r = 2$.

Variant	Rank	Alpha	Trainable params	Base %	Adapter
QLoRA r8	8	16	5,046,272	0.0663%	~31 MB
QLoRA r16	16	32	10,092,544	0.1325%	~50 MB

Table 3: *Rank experiment. Percentages use the logical 7.615B base-model parameter count as denominator.*

QLoRA keeps the base weights frozen and quantized to 4-bit NF4 while LoRA A/B matrices remain trainable; BF16 is used for computation and double quantization is enabled. Quantization is therefore a storage/compute representation for the frozen base, not a statement that the 4-bit base weights themselves are updated.

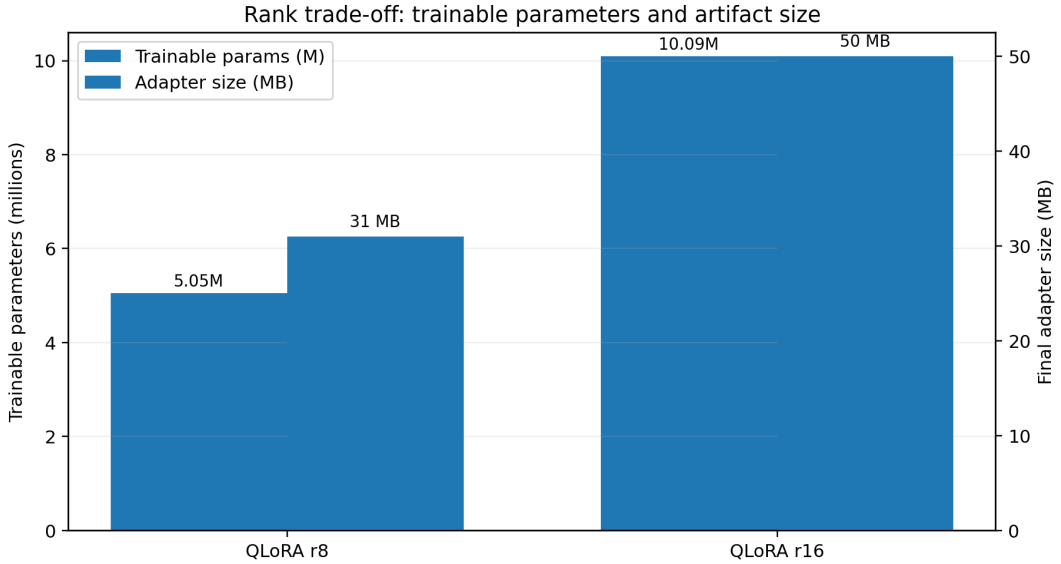


Figure 1: *Doubling rank doubles the trainable LoRA parameter count while validation changes only marginally.*

6. Training methodology and resource use

Both rank variants use the same explicit training configuration: one epoch, batch size 4, gradient accumulation 4 (effective batch 16), learning rate 2×10^{-4}, zero weight decay, max grad norm 1, warmup ratio 0.03, gradient checkpointing, BF16 compute, and seed 42.

The training implementation uses an explicit PyTorch loop rather than hiding mechanics behind Trainer/TRL. The sequence is: batch → forward → loss → gradient-accumulation scaling → backward → clipping → optimizer step → scheduler → adapter checkpoint.

Training metric	QLoRA r8	QLoRA r16
Train examples	13,334	13,334
Optimizer steps	834	834
Training time	76.92 min	77.23 min
Peak GPU allocated	18.18 GiB	18.25 GiB
Peak GPU reserved	22.40 GiB	22.72 GiB
Checkpoint cadence	200 steps	200 steps

Table 4: *Full-training resource summary on the NVIDIA A30 24 GB node.*

6.1 Memory evidence

BF16 base loading allocated roughly 14.19 GiB, while 4-bit loading allocated roughly 5.18 GiB: about 9.01 GiB, or 63.5%, lower loading allocation. In smoke training, QLoRA peaked at 8.81 GiB compared with 15.52 GiB for BF16 LoRA, a project-specific reduction of about 43.2%.

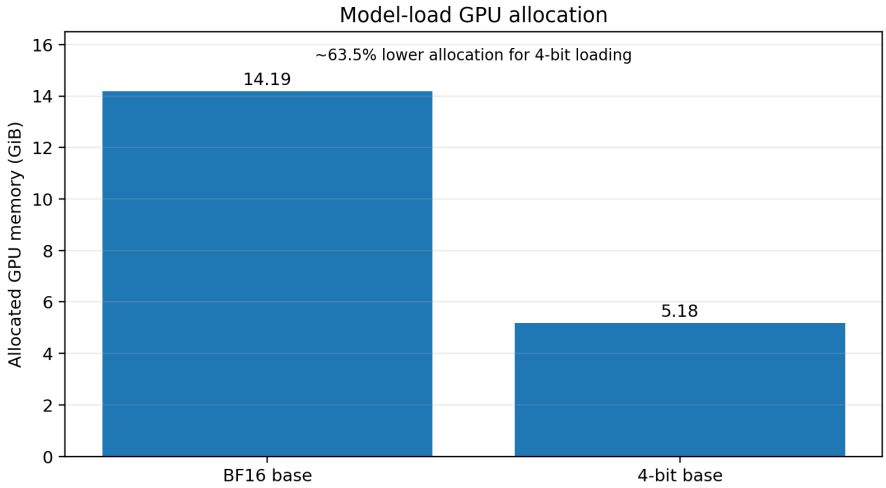


Figure 2: *Observed GPU allocation for BF16 versus 4-bit base-model loading.*

These memory results are specific to this hardware/software stack and should not be interpreted as universal savings percentages.

7. Validation comparison and adapter selection

Validation was run on the same 1,471 filtered examples for the base model and both QLoRA variants.

Model	Validation loss	Perplexity	Loss change vs base
Base	1.654063	5.2282	–
QLoRA r8	1.592670	4.9169	-3.71%
QLoRA r16	1.591850	4.9128	-3.76%

Table 5: *Full-validation results.*

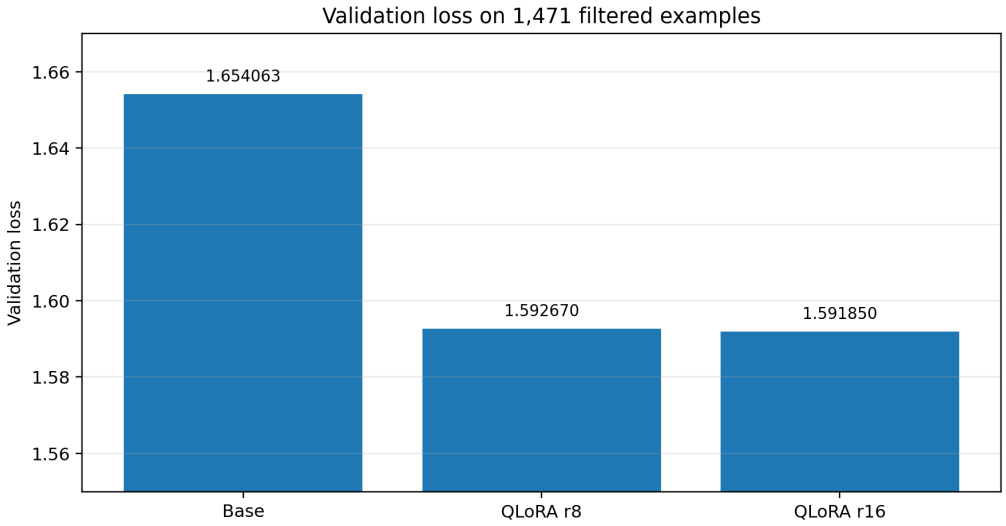


Figure 3: *Validation loss comparison. The r8/r16 difference is only 0.000820.*

r16 is numerically slightly better on loss/perplexity, but the difference is tiny relative to the doubled trainable LoRA parameter count and larger adapter artifact. The selected production-demo adapter is therefore r8, reinforced by the qualitative comparison below. This is an engineering choice for this experiment, not a general claim that rank 8 is superior.

8. Qualitative generation comparison

The comparison artifact contains 24 deterministic examples: three examples from each of eight Dolly categories, with base, r8, r16, reference, and prompt fields preserved.

Observed behavior in this sample:

- the base model continued into extra `### Instruction` / `### Response` blocks in 5/24 examples;
- r8 and r16 showed this formatting failure in 0/24 examples;
- both QLoRA variants improved instruction-following and response-format behavior;
- individual regressions still existed, including one extraction example where the base response was preferable;
- r16 showed more repetitive degeneration in several inspected generations, while r8 and r16 were otherwise often similar.

This evidence supports r8 as the selected demo/runtime adapter because it combines nearly identical validation performance with fewer trainable parameters, a smaller artifact, and slightly more stable behavior in the inspected sample.

9. Inference benchmark matrix

The controlled benchmark uses the same 24 prompts, deterministic generation, exactly 128 generated tokens per prompt, and therefore 3,072 generated tokens for every runtime mode.

Runtime	Load	Mean	P95	tok/s	Peak VRAM
Base 4-bit	3.5288 s	3.7256 s	3.7656 s	34.357	5.3837 GiB
QLoRA r8, 4-bit + PEFT	3.3908 s	6.1402 s	6.2051 s	20.846	5.3993 GiB
Base BF16	3.6132 s	2.9426 s	2.9782 s	43.499	14.2712 GiB
BF16 + PEFT	3.3062 s	4.3147 s	4.3504 s	29.6659	14.2892 GiB
Merged BF16	3.3507 s	2.9356 s	2.9677 s	43.6031	14.3464 GiB

Table 6: *Controlled inference benchmark on the A30.*

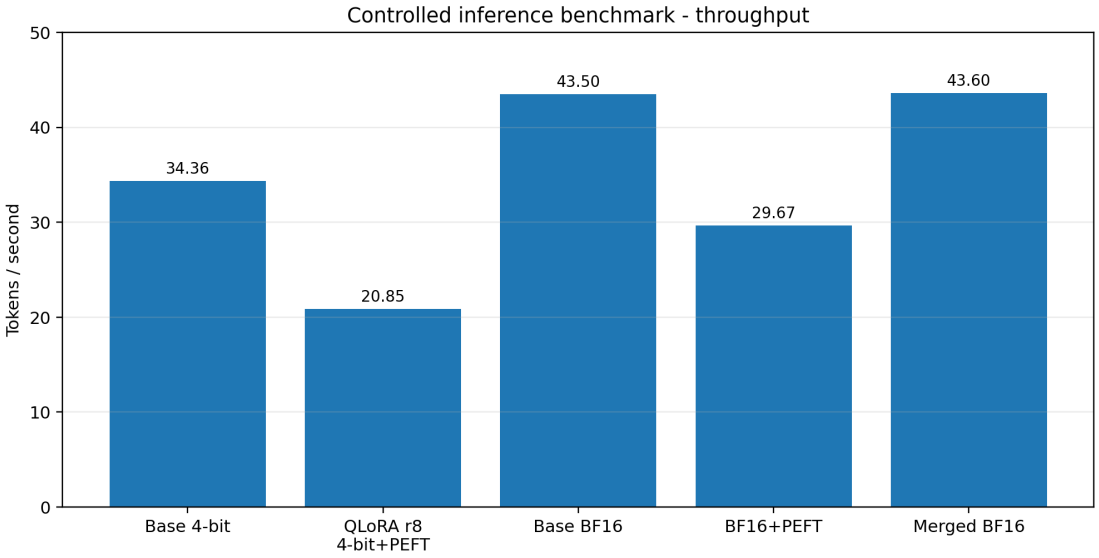


Figure 4: *Throughput comparison. 4-bit is memory-efficient but not automatically faster on this GPU.*

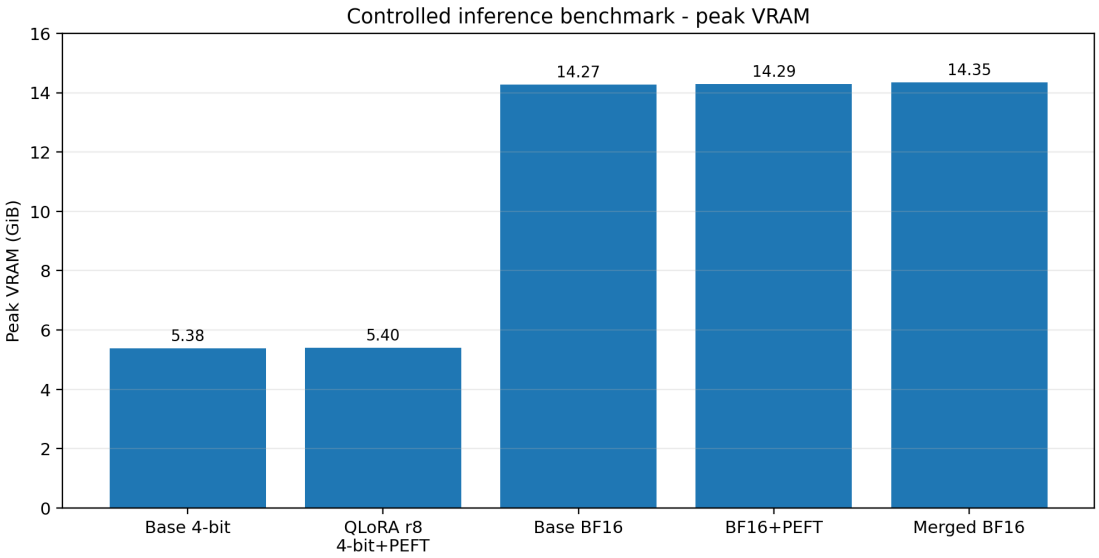


Figure 5: *Peak VRAM comparison across inference representations.*

The practical deployment trade-off is clear: 4-bit + PEFT uses about 5.4 GiB peak VRAM but reaches 20.85 tok/s, while merged BF16 uses about 14.35 GiB but reaches 43.60 tok/s. Relative to BF16 + PEFT, merged BF16 lowers latency by roughly 32% and increases throughput by roughly 47% with almost unchanged VRAM.

10. Merge tests and numerical equivalence

Both 4-bit and BF16 merge smoke tests succeeded technically, but exact generated text changed before versus after merge. For the BF16 logit check, the immediate next-token argmax remained identical while finite numerical differences were measurable: maximum absolute logit difference 0.25 and mean absolute difference 0.0278659.

This is consistent with autoregressive generation: small finite-precision changes can eventually alter a later argmax and then diverge the remaining continuation. The project therefore does not claim bit-exact generation equivalence after merge.

11. Production-oriented system architecture

The selected runtime path keeps the application layer independent of a specific GPU host. The browser calls FastAPI; FastAPI invokes a compiled LangGraph workflow; LangGraph calls a ModelService abstraction; the service can be a mock backend or the real QLoRA r8 model. Logging and Prometheus-style metrics observe the runtime path.

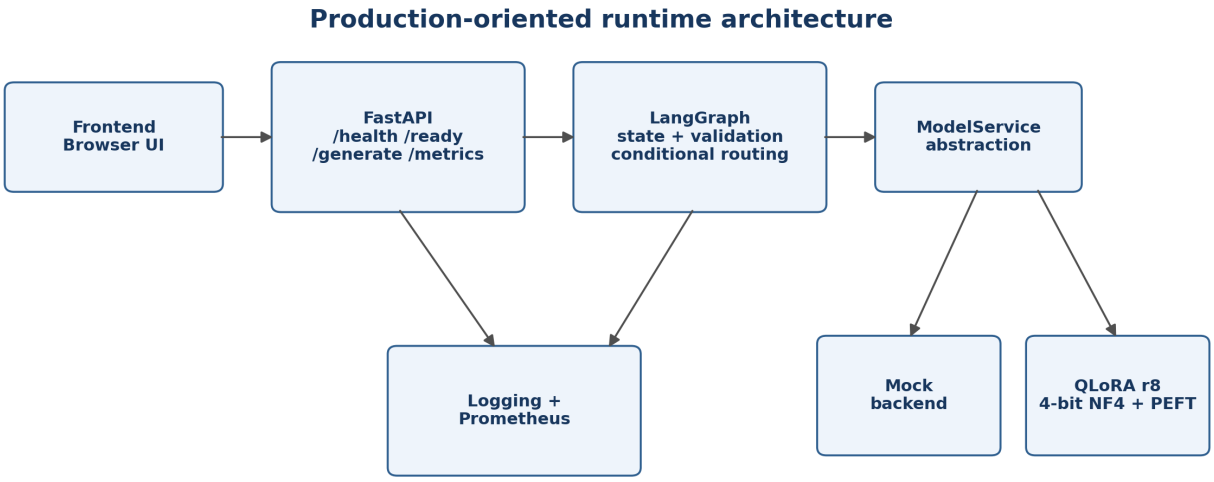


Figure 6: *Final application architecture.*

The QLoRA runtime loads the local Qwen2.5-7B base in 4-bit NF4 with BF16 compute and double quantization, attaches `results/qlora_r8/final_adapter` through PEFT, and runs deterministic generation. Application settings are controlled through environment variables rather than notebook state.

12. FastAPI behavior and mock-mode development

The backend exposes `/health`, `/ready`, `/model-info`, `/generate`, and `/metrics`. Health is process liveness; readiness checks whether the generation workflow has been initialized. The request schema requires a non-empty instruction and accepts optional context. The response returns text, model mode, latency, and generated-token count.

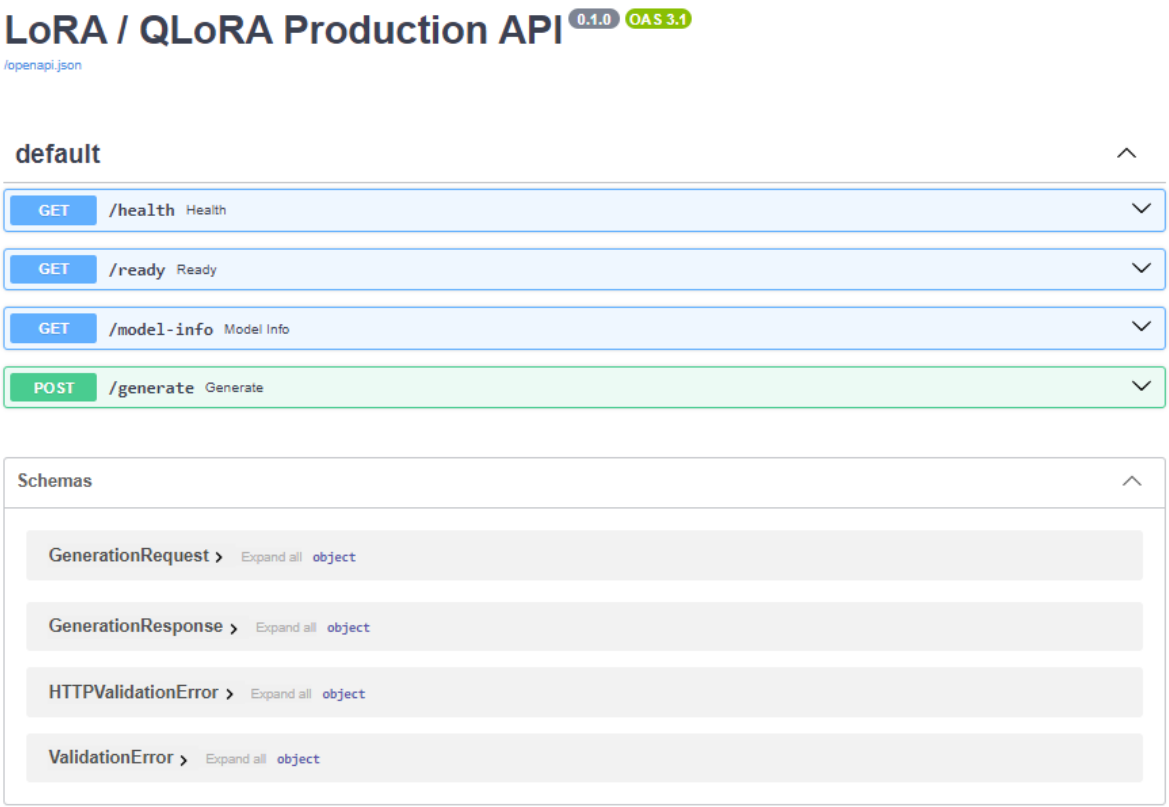
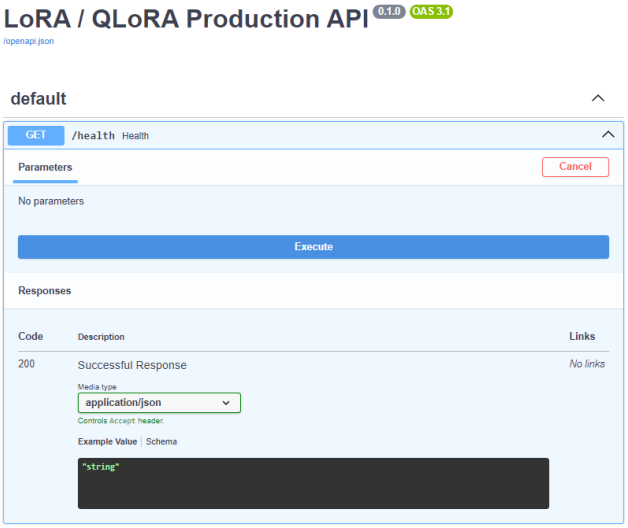
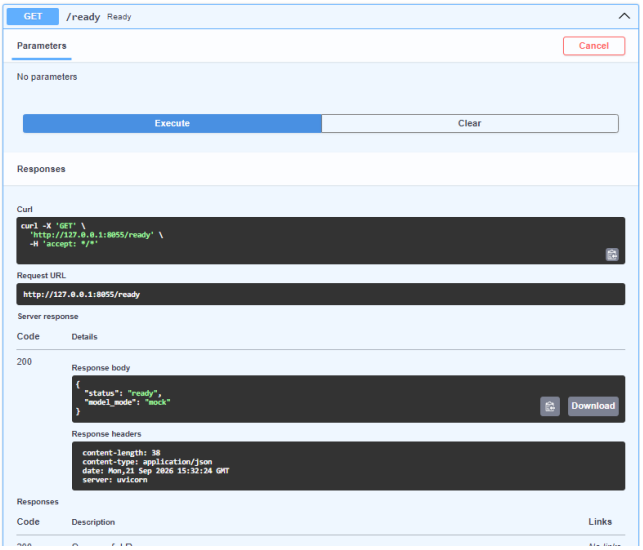


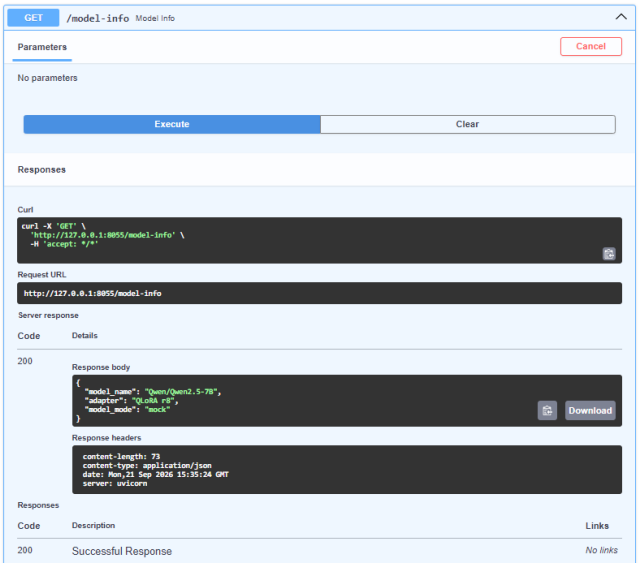
Figure 7: *Swagger overview of the final API surface.*



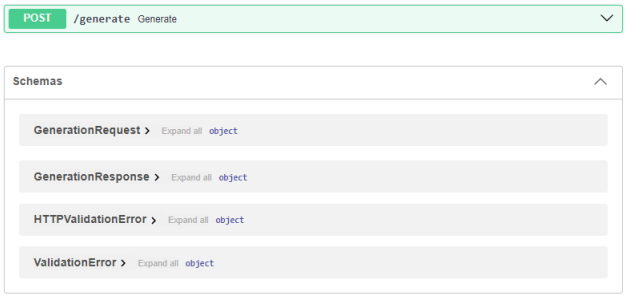
(a) *Health endpoint.*



(b) *Readiness endpoint.*



(c) *Model metadata.*



(d) *Mock generation schema.*

Figure 8: *Mock-mode endpoint validation before loading the 7B checkpoint.*

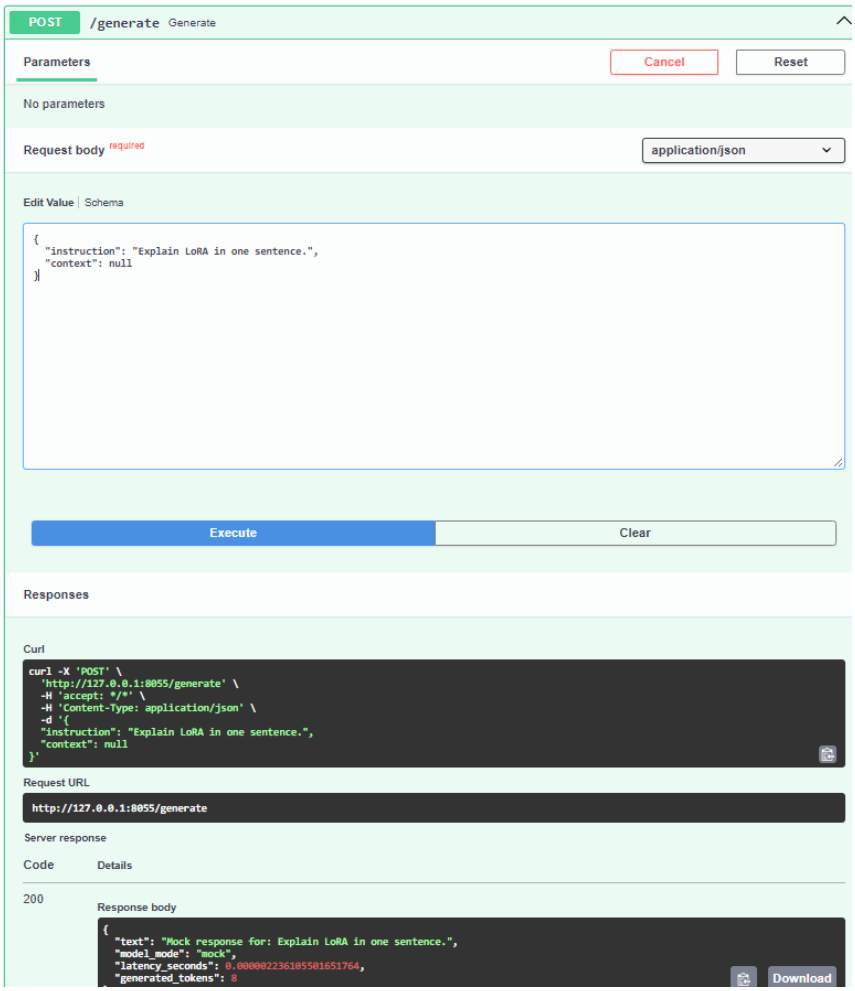


Figure 9: *Mock response proving the request/response contract before GPU integration.*

13. Input validation and real QLoRA API integration

Pydantic validation rejects an empty instruction with HTTP 422 before model inference. This separates client-schema errors from internal model/runtime failures.

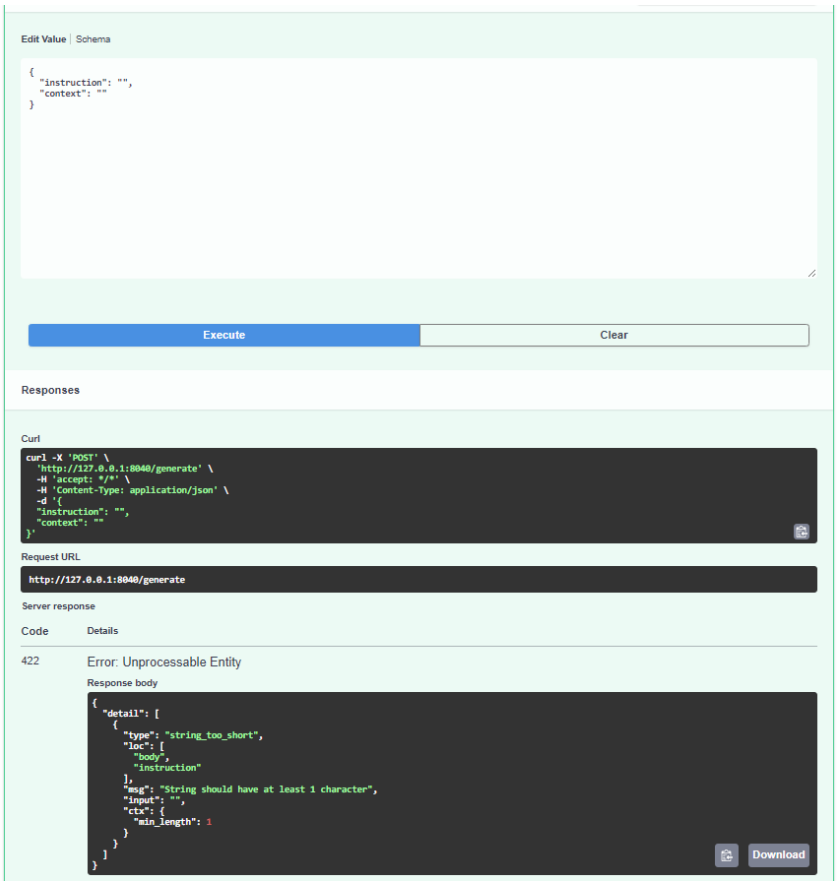
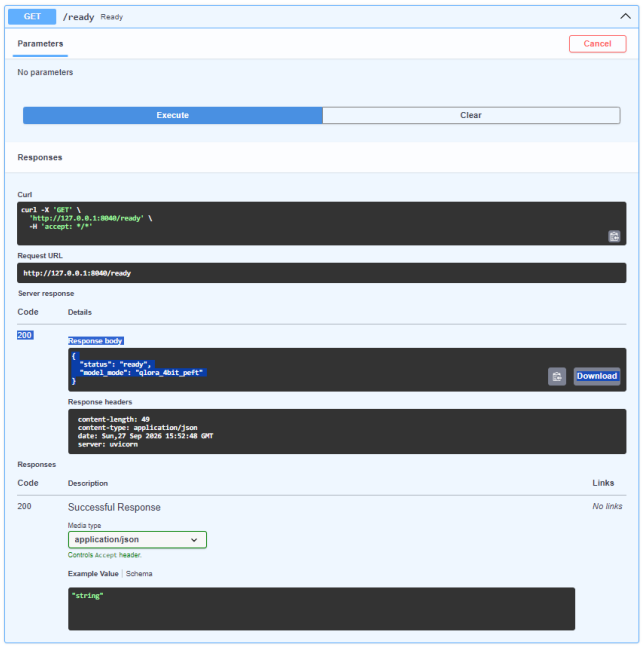
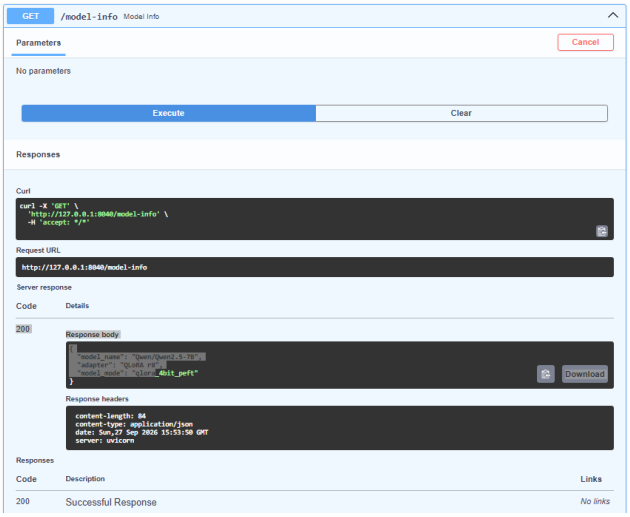


Figure 10: *Structured HTTP 422 validation error for an empty instruction.*

With `MODEL_MODE=qlora_4bit_peft`, the real service is loaded on CUDA. Readiness and model metadata expose the active runtime mode and selected adapter.



(a) *Real-model readiness.*



(b) *Real model information.*

Figure 11: *Real QLoRA runtime metadata exposed through FastAPI.*

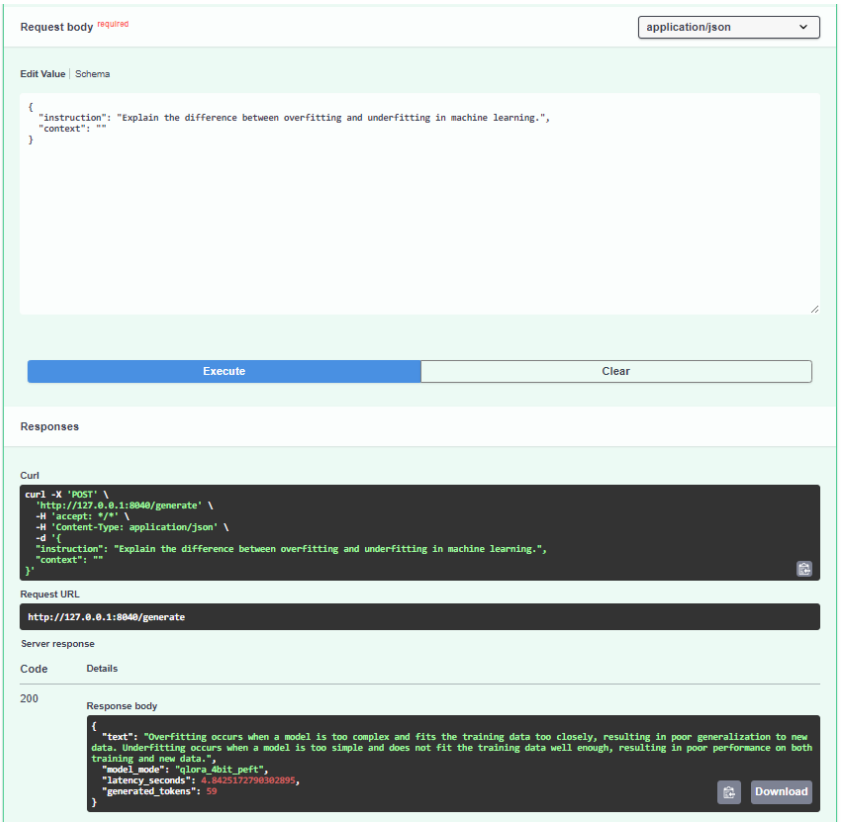


Figure 12: *Real QLoRA generation through `POST /generate`. The response includes generated text, runtime mode, latency, and token count.*

14. LangGraph orchestration

The workflow is intentionally small but meaningful. A generation node calls the selected ModelService; a validation node checks whether the generated text is non-empty; conditional routing either terminates successfully or enters an explicit invalid-output path that raises an error.

```
generate
   |
   v
validate_output
   |
   +---- non-empty ----> END
   |
   +---- empty --------> invalid_output -> error
```

This design provides explicit state, nodes, edges, and conditional routing without using LangGraph as a decorative wrapper.

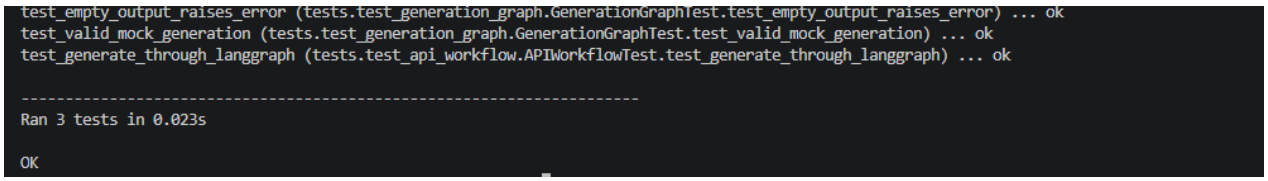


Figure 13: *Initial LangGraph validation tests, including valid generation and empty-output routing.*

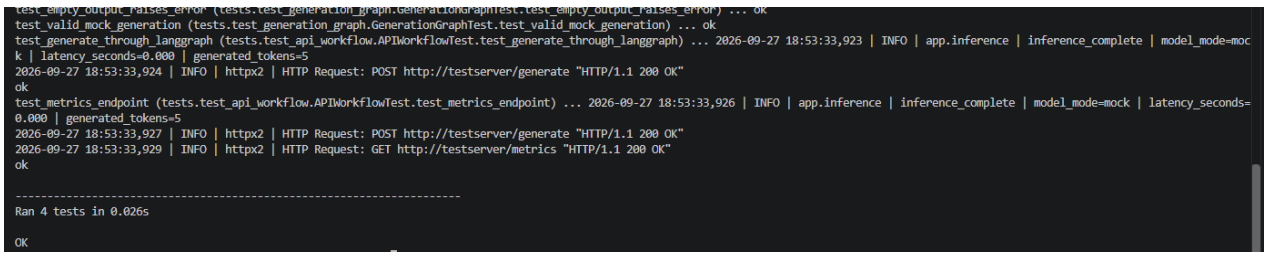


Figure 14: *Expanded workflow/API tests after LangGraph and metrics integration.*

15. Frontend and end-to-end user flow

The frontend is deliberately lightweight and dependency-free: a static HTML/CSS/JavaScript interface served separately from the API. It displays API status, model/adapter metadata, instruction and optional context inputs, model response, latency, generated-token count, and runtime mode.

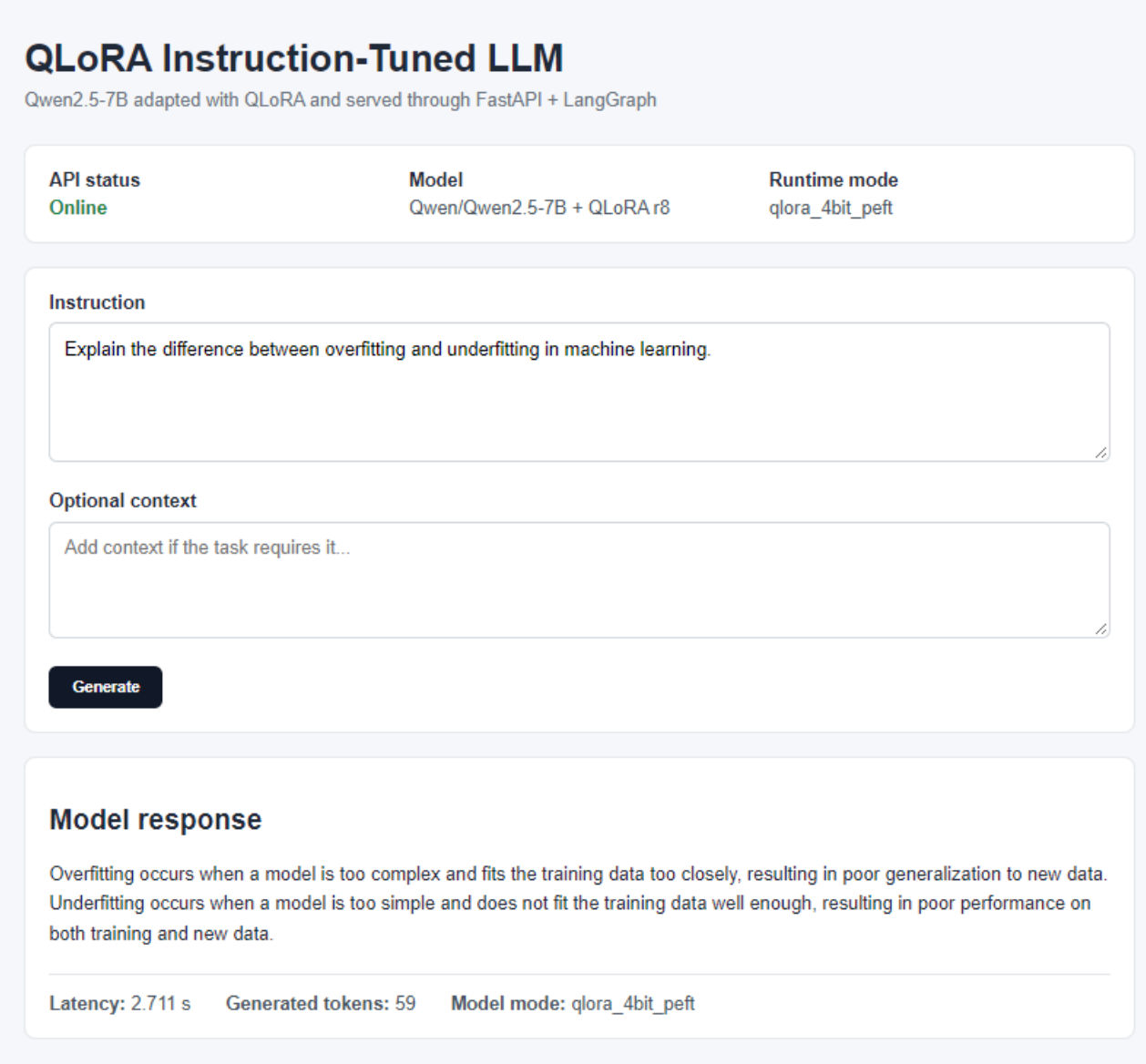


Figure 15: *Browser frontend performing real QLoRA inference through the FastAPI + LangGraph stack.*

The displayed request asks for the difference between overfitting and underfitting. The validated run returned a coherent response with 59 generated tokens and approximately 2.7 seconds application-reported inference latency. This single request is operational evidence rather than a replacement for the controlled benchmark matrix.

16. Logging and observability

Application logging records successful inference completion with model mode, latency, and generated-token count. Prometheus-style metrics expose request count by status/model mode, inference-latency histogram, and generated-token totals.

A real QLoRA request produced the following observed metric values: one successful request, one latency observation with sum 2.7702 seconds, and 59 generated tokens.

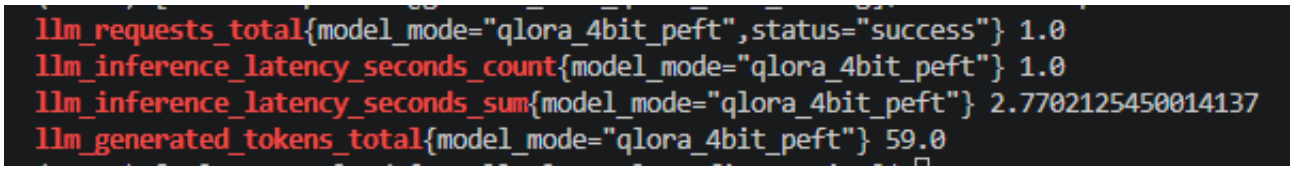


Figure 16: *Prometheus exposition after a successful real QLoRA request.*

This is intentionally lightweight runtime observability. The project does not claim an external Prometheus/Grafana deployment, alerting stack, or production SLO program.

17. Lifecycle, readiness, and testing

Model initialization was moved into FastAPI lifespan startup. The app stores the ModelService and compiled LangGraph workflow in application state, allowing readiness to reflect whether the runtime has actually been initialized rather than returning a static success response.

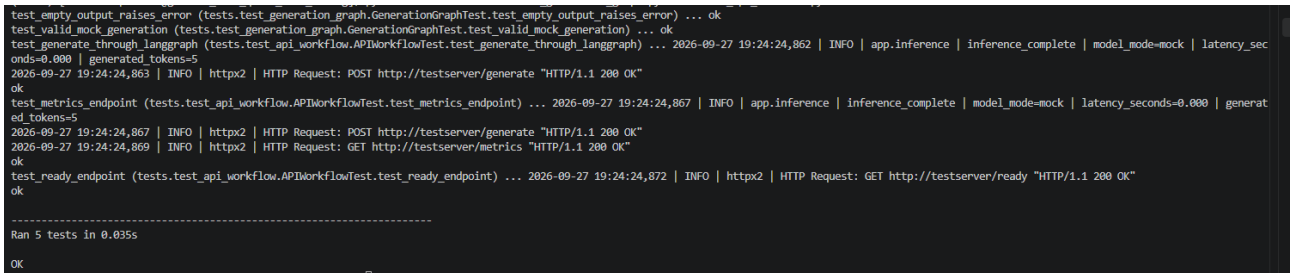


Figure 17: *Five-test suite after lifecycle-based initialization and readiness validation.*

The final suite adds an opt-in real GPU API integration test. Lightweight tests use mock mode and do not load the 7B checkpoint; the GPU test sends a real HTTP request to the running API and verifies status 200, active `qlora_4bit_peft` mode, non-empty text, and a positive generated-token count.

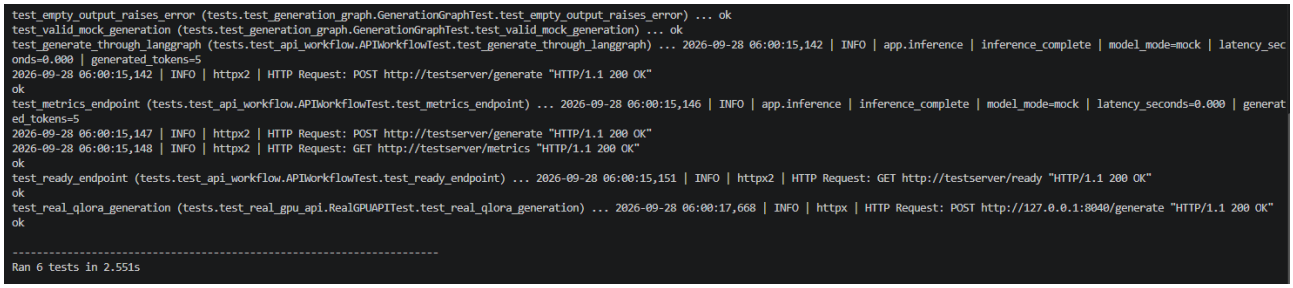


Figure 18: *Final six-test suite, including a successful real GPU end-to-end `POST /generate`.*

The six final checks cover valid/invalid LangGraph routing, mock API generation through the graph, metrics, readiness, and real QLoRA API inference.

18. Containerization and deployment readiness

The repository includes both a backend Dockerfile and a Singularity definition. Large model/data artifacts are not baked into the application image; model mode, base-model path, adapter path, and generation settings are environment-configurable.

Container validation is reported conservatively:

- Docker was not installed on the university cluster, so a Docker build was not claimed there.
- Singularity CE 4.3.2 was available, but local `-fakeroot` build failed because the cluster account had no valid fakeroot mapping.
- remote Singularity build was available only after external authentication; this was intentionally not configured.
- final Docker build/run validation is therefore deferred to the local Windows environment where Docker is available.

This preserves useful deployment artifacts without misrepresenting an unexecuted container build as validated.

19. Reproducibility and repository packaging

The environment was checked with `pip check` and returned "No broken requirements found." A final `requirements-lock.txt` snapshot was generated after installing the application dependencies.

Component	Validated value
Scheduler	SLURM
GPU	NVIDIA A30 24 GB
Python	3.11.11
PyTorch	2.11.0+cu128
Transformers	5.17.0
Datasets	5.0.1
Accelerate	1.15.0
PEFT	0.21.0
bitsandbytes	0.50.2
FastAPI	0.141.1
Uvicorn	0.53.0
LangGraph	1.2.12
prometheus-client	0.26.0

Table 7: *Validated software environment.*

The complete working project is roughly 22 GB because it contains the local base model, processed data, checkpoints/results, and environment/cache material. Normal Git history should contain source code, configs, scripts, SLURM wrappers, tests, README/docs, small benchmark summaries, and deployment configuration, while excluding model weights, large datasets/checkpoints, `.venv`, secrets, and massive logs.

19.1 Reproducibility boundaries

Two implementation nuances are documented rather than hidden:

- the final partial gradient-accumulation group is still divided by the full accumulation factor of 4; this affects only the final optimizer step and was identical for r8/r16;
- the DataLoader relied on the global PyTorch RNG rather than a dedicated generator, so rank-dependent LoRA initialization may mean exact stochastic sample ordering is not bit-identical even with the same seed.

The rank study is controlled in its explicit hyperparameters, but the project does not claim bitwise deterministic CUDA training or bit-exact stochastic ordering.

20. Limitations

- One base model, one instruction dataset, one primary GPU architecture, and two LoRA ranks were studied.
- Validation loss/perplexity and 24 qualitative examples do not constitute a complete downstream task benchmark.
- The real API serves one model path rather than a multi-tenant inference platform.
- No long-duration load testing, concurrency tuning, autoscaling, authentication, rate limiting, or SLO validation is claimed.
- Metrics are intentionally lightweight; no external monitoring deployment is claimed.
- Docker/Singularity definitions are implementation artifacts, but the final Docker image build/run still requires local validation.
- Large model weights and adapters are external runtime artifacts rather than normal Git content.
- Merge equivalence is numerical/functional rather than bit-exact generation equivalence.

21. What would change at production scale

A production deployment would extend this design with dedicated inference serving, request concurrency controls, authentication and authorization, rate limits, structured request IDs/tracing, centralized logs, external metrics/alerts, GPU scheduling and autoscaling, artifact registry/versioning, canary releases, rollback, security scanning, and systematic load/SLO testing. Those are intentionally described as next steps rather than implied as already deployed.

22. Conclusion

The project demonstrates a complete parameter-efficient LLM engineering workflow: dataset analysis, explicit supervision construction, QLoRA memory optimization, controlled rank comparison, validation and qualitative evaluation, deployment-mode benchmarking, merge analysis, FastAPI/LangGraph serving, browser interaction, observability, lifecycle management, and real GPU end-to-end testing.

The most useful technical conclusion is that optimization objectives differ across training and inference. QLoRA makes adaptation feasible within a 24 GB GPU budget, but the fastest runtime on this A30 is not the lowest-memory runtime. Similarly, doubling LoRA rank offers negligible validation improvement here while doubling trainable adapter parameters. The final r8 choice therefore reflects a measured efficiency/quality trade-off rather than a default preference.

The project remains bounded and transparent about what is and is not validated. That boundary is part of the engineering evidence: results, operational behavior, reproducibility limitations, and deployment constraints are documented rather than hidden.

Appendix A. Key commands

Real GPU API

```
MODEL_MODE=qlora_4bit_peft \
python -m uvicorn app.main:app --host 127.0.0.1 --port 8040
```

Frontend

```
python -m http.server 8050 --bind 127.0.0.1 --directory frontend
```

Full test suite including GPU integration

```
RUN_GPU_INTEGRATION=1 python -m unittest \
  tests/test_generation_graph.py \
  tests/test_api_workflow.py \
  tests/test_real_gpu_api.py -v
```

Training

```
python scripts/train_qlora.py --config configs/qlora_r8.yaml
python scripts/train_qlora.py --config configs/qlora_r16.yaml
```

Appendix B. Evidence inventory

The report incorporates the complete captured screenshot inventory: health, readiness, model information, mock generation, Swagger overview, schema validation, real QLoRA readiness/model metadata/generation, LangGraph tests, frontend inference, metrics, lifecycle/readiness tests, and the final real-GPU end-to-end test suite.